

Продвинутые техники CSS

Мощные инструменты для создания сложной верстки без JavaScript

Докладчики: Барабанов Илья, Волков Иван

CSS-счётчики: автоматическая нумерация

CSS-счётчики позволяют управлять нумерацией элементов автоматически, без использования JavaScript. Это особенно полезно для создания оглавлений, нумерованных списков и сложной документации.



counter-reset

Создает или сбрасывает счётчик. Принимает имя счётчика и начальное значение (по умолчанию 0).



counter()

Вывод значения через свойство content в псевдоэлементах ::before или ::after.



counter-increment

Увеличивает значение счётчика. Принимает имя счётчика и шаг (по умолчанию 1).

Пример 1: Акцент на `counter-reset` (Начальное значение)

CSS

```
.container {  
  counter-reset: step 10;  
  
}  
  
.item {  
  counter-increment: step;  
  margin: 5px 0;  
}  
  
.item::before {  
  content: counter(step) ". ";  
  color: red;  
  font-weight: bold;  
}
```

Пример с counter-reset: 10

- 11. Элемент 1 (станет 11)
- 12. Элемент 2 (станет 12)
- 13. Элемент 3 (станет 13)

Изменения в counter-reset

Здесь мы меняем только начальное число. Если поставить **10**, нумерация начнется с 11 (так как инкремент +1).

HTML

```
<div class="container">  
  <div class="item">Элемент 1 (станет  
11)</div>  
  <div class="item">Элемент 2 (станет  
12)</div>  
  <div class="item">Элемент 3 (станет  
13)</div>  
</div>
```

Пример 2: Акцент на counter-increment (Шаг увеличения)

CSS

```
.container {  
  counter-reset: step;  
}  
  
.item {  
  counter-increment: step 2;  
  margin: 5px 0;  
}  
  
.item::before {  
  content: counter(step) ". ";  
  color: green;  
  font-weight: bold;  
}
```

Пример с counter-increment: 2

- 2. Элемент 1 (значение 2)
- 4. Элемент 2 (значение 4)
- 6. Элемент 3 (значение 6)

Изменения в counter-increment

Здесь мы меняем шаг. Обычно он равен 1, но если написать **step 2**, то числа будут прыгать через один (1, 3, 5...).

HTML

```
<div class="container">  
  <div class="item">Элемент 1 (значение 2)</div>  
  <div class="item">Элемент 2 (значение 4)</div>  
  <div class="item">Элемент 3 (значение 6)</div>  
</div>
```

Пример 3: Акцент на `counter()` (Формат вывода)

CSS

```
.container {  
  counter-reset: step;  
}  
  
.item {  
  counter-increment: step;  
  margin: 5px 0;  
}  
  
.item::before {  
  
  content: "Раздел " counter(step, upper-roman) ": ";  
  
  color: purple;  
  font-weight: bold;  
  text-transform: uppercase;  
}
```

Пример с форматированием counter()

РАЗДЕЛ I: Текст пункта

РАЗДЕЛ II: Текст пункта

РАЗДЕЛ III: Текст пункта

РАЗДЕЛ IV: Текст пункта

РАЗДЕЛ V: Текст пункта

Обратите внимание: цифры стали римскими (I, II, III...)

Изменения в counter

Здесь мы меняем только то, как выглядит цифра. Можно добавить текст до или после числа, или даже изменить систему счисления (например, на римские цифры).

HTML

```
<div class="container">  
  <div class="item">Текст пункта</div>  
  <div class="item">Текст пункта</div>  
  <div class="item">Текст пункта</div>  
  <div class="item">Текст пункта</div>  
  <div class="item">Текст пункта</div>  
</div>
```

Многоколоночный текст

Разбивка блочного контента на несколько колонок для создания газетной верстки.

1 column-count

Задаёт количество колонок.
Принимает целые числа или auto.

2 column-gap

Расстояние между колонками.
Единицы измерения: px, em, %
или normal.

3 columns

Короткая запись, объединяющая column-width и column-count (например, columns: 200px 3).

`column-count` (Количество колонок)

CSS

```
.container {  
  column-count: 2;  
}  
  
.item {  
  margin: 5px 0;  
}
```

Пример `column-count: 3`

Элемент 1

Элемент 3

Элемент 2

Элемент 4

Изменения в `column-count`

Разбиваем контент на 2
колонки

HTML

```
<div class="container">  
  
  <div class="item">Элемент 1</div>  
  <div class="item">Элемент 2</div>  
  <div class="item">Элемент 3</div>  
  <div class="item">Элемент 4</div>  
  
</div>
```

CSS

```
.container {  
  
column-count: 2;  
  
column-gap: 40px;  
  
border: 1px solid #ccc;}  
  
.item { margin: 5px 0;}
```

`column-count` (Количество колонок)

Пример `column-gap: 40px`

Текст левой колонки	Текст правой колонки
Текст левой колонки	Текст правой колонки
Текст левой колонки	Текст правой колонки

Изменения в `column-gap`

Отступ 40 пикселей между колонками

HTML

```
<div class="container">  
  <div class="item">Текст левой колонки</div>  
  <div class="item">Текст левой колонки</div>  
  <div class="item">Текст левой колонки</div>  
  <div class="item">Текст правой колонки</div>  
  <div class="item">Текст правой колонки</div>  
  <div class="item">Текст правой колонки</div>  
</div>
```


`column-count` (Количество колонок)

CSS

```
.container {  
  columns: 2 20px;  
  border: 1px solid #ccc;  
  padding: 10px;  
}  
  
.item {  
  background: #fff3e0;  
  margin: 5px 0;  
  padding: 5px;  
  break-inside: avoid;  
}
```

Пример `columns: 3 20px`

Пункт 1	Пункт 3
Пункт 2	Пункт 4

Изменения в `columns`

`columns: [количество] [отступ]`
Поставили 2 колонки, 20 пикселей отступ

HTML

```
<div class="item">Пункт 1</div>  
  <div class="item">Пункт 2</div>  
  <div class="item">Пункт 3</div>  
  <div class="item">Пункт 4</div>
```

Клиппинг: обрезание элементов

Свойство `clip-path` скрывает часть элемента, оставляя видимой только определенную область или фигуру. Это мощный инструмент для создания нестандартных форм без использования изображений.

`circle()`

Круглая область обрезания

`ellipse()`

Эллиптическая область

`polygon()`

Многоугольная область

`inset()`

Внутреннее обрезание

`path()`

Сложные SVG-контуры

`url()`

Ссылка на SVG-маску

Пример circle()

CSS

```
.box {  
  width: 100px;  
  height: 100px;  
  background: linear-gradient(45deg,  
    #ff6b6b, #feca57);  
  
  clip-path: circle(50% at center);  
}
```

Пример circle()



Квадрат обрезан до круга

Изменения в circle()

Круг: радиус 50% от центра

HTML

```
<div class="box"></div>
```

Пример ellipse()

CSS

```
.box {  
  width: 100px;  
  height: 100px;  
  background: linear-gradient(45deg,  
    #5f27cd, #48dbfb);  
  /* */  
  clip-path: ellipse(70% 40% at center);  
}
```

Пример ellipse()



Квадрат обрезан до овала

Изменения в ellipse()

Эллипс: радиус X 70%, радиус Y 40%

HTML

```
<div class="box"></div>
```

Пример polygon()

CSS

```
.box {  
  width: 100px;  
  height: 100px;  
  background: linear-gradient(45deg,  
    #1dd1a1, #10ac84);  
  
  clip-path: polygon(50% 0%, 0%  
    100%, 100% 100%);  
}
```

HTML

```
<div class="box"></div>
```

Пример polygon() — треугольник



Координаты: верх-центр, низ-лево, низ-право

Изменения в polygon()

Треугольник: 3 точки координат
50% 0%, 0% 100%, 100% 100%

Пример inset()

CSS

```
.box {  
  width: 100px;  
  height: 100px;  
  background: linear-gradient(45deg,  
    #ff9ff3, #f368e0);  
  
  clip-path: inset(25% round 20px);  
}
```

Пример inset() — прямоугольник внутри



Обрезано по 25% со всех сторон + скругление

Изменения в inset()

Врезка: по 25% с каждой стороны + скругление

HTML

```
<div class="box"></div>
```

Пример path()

CSS

```
.box {  
  width: 200px;  
  height: 200px;  
  background: linear-gradient(45deg,  
    #00d2d3, #54a0ff);  
  
  clip-path: path('M 0 50 Q 50 0 100 50  
    T 200 50 V 200 H 0 Z');  
}
```

Пример path() — волна



SVG-путь для сложной формы

Изменения в path()

SVG-путь: волнистая линия

HTML

```
<div class="box"></div>
```

Пример url()

CSS

```
.box {  
  width: 200px;  
  height: 200px;  
  background: linear-gradient(45deg,  
    #f0932b, #eb4d4b);  
  /* */  
  clip-path: url(#star);  
}
```

Пример url() — звезда через SVG



Изменения в url()

Ссылка на SVG-маску

HTML

```
<div class="box"></div>  
  
<!-- Скрытый SVG с маской -->  
<svg width="0" height="0">  
  <defs>  
    <clipPath id="star">  
      <polygon points="100,10 40,180 190,60 10,60 160,180"/>  
    </clipPath>  
  </defs>  
</svg>
```


Пример

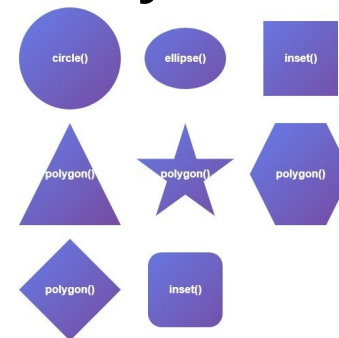
```
<!DOCTYPE html>
<html>
<head>
<style>
.container { display: flex; flex-wrap: wrap; gap: 20px; padding: 20px;
font-family: Arial;
}

.box {
width: 150px; height: 150px;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); transition: all 0.3s;
text-align: center; line-height: 50px; color: white;
font-weight: bold;
}

.box:hover {
transform: scale(1.05);
}

.circle { clip-path: circle(50%); }
.ellipse { clip-path: ellipse(40% 30%); }
.inset { clip-path: inset(20px); }
.triangle { clip-path: polygon(50% 0%, 0% 100%, 100% 100%); }
.star { clip-path: polygon(50% 0%, 61% 35%, 98% 35%, 68% 57%, 79% 91%, 50% 70%, 21% 91%, 32% 57%, 2% 35%, 39% 35%); }
.hexagon { clip-path: polygon(25% 0%, 75% 0%, 100% 50%, 75% 100%, 25% 100%, 0% 50%); }
.diamond { clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%); }
.rounded-inset { clip-path: inset(20px round 20px); }
</style>
</head>
<body>
<div class="container">
<div class="box circle">circle</div>
<div class="box ellipse">ellipse</div>
<div class="box inset">inset</div>
<div class="box triangle">треугольник</div>
<div class="box star">звезда</div>
<div class="box hexagon">шестиугольник</div>
<div class="box diamond">ромб</div>
<div class="box rounded-inset">inset</div>
</div>
</body>
</html>
```

Результат



CSS фигуры: полная таблица сравнения

Функция	Описание	Синтаксис	Пример
<code>circle()</code>	Круг	<code>circle(радиус at x y)</code>	<code>circle(50%)</code> <code>circle(100px at 30% 40%)</code>
<code>ellipse()</code>	Эллипс	<code>ellipse(радиусX радиусY at x y)</code>	<code>ellipse(40% 30%)</code> <code>ellipse(80px 50px at 50% 50%)</code>
<code>inset()</code>	Прямоугольник с отступами	<code>inset(верх право низ лево round радиус)</code>	<code>inset(20px)</code> <code>inset(10px 20px 30px 40px round 15px)</code>
<code>polygon()</code>	Многоугольник	<code>polygon(x1 y1, x2 y2, x3 y3, ...)</code>	<code>polygon(50% 0%, 0% 100%, 100% 100%)</code> <code>polygon(0% 0%, 100% 0%, 50% 100%)</code>
<code>path()</code>	SVG путь	<code>path("команды_пути")</code>	<code>path("M 100,30 C 130,0 170,0 200,30 Z")</code> <code>path("M 0,0 L 100,0 L 50,100 Z")</code>
<code>url()</code>	Ссылка на SVG маску	<code>url(#id_маски)</code>	<code>url(#heartClip)</code> <code>url(clipPath.svg#myClip)</code>

Маскирование: творческое обрезание

Как работает mask

Свойство `mask` использует изображение (обычно с прозрачным фоном или градиент) для скрытия части элемента. В отличие от `clip-path`, где видимость определяется формой, здесь видимость зависит от прозрачности маски.

Там, где маска непрозрачна — элемент виден. Там, где маска прозрачна — элемент скрыт.

Значения `mask`

- `url()` — путь к изображению
- `linear-gradient()` — линейный градиент
- `radial-gradient()` — радиальный градиент
- `none` — отсутствие маски



Мощность: `mask` — это шорткат, объединяющий `mask-image`, `mask-mode`, `mask-repeat`, `mask-position` и другие свойства.

Кастомизация маркера списка

Полный контроль над внешним видом "буллитов" в списках `<ul>` и `<ol>`.



`::marker`

Псевдоэлемент для прямой стилизации маркера (цвет, шрифт, размер)



`list-style-type`

Тип маркера: `disc`, `circle`, `square`, `decimal` и другие



`list-style-image`

Пользовательское изображение вместо стандартного маркера



`list-style` — короткая запись, объединяющая `list-style-type`, `list-style-position` и `list-style-image`

Пример ::marker

CSS

```
.styled-list li::marker {  
    color: #ff6b6b;  
    font-size: 20px;  
    font-weight: bold;  
    content: "✓ ";  
}  
  
.styled-list li {  
    margin: 10px 0;  
    font-size: 16px;  
}
```

Пример ::marker

- ✓ Первый пункт
- ✓ Второй пункт
- ✓ Третий пункт
- ✓ Четвертый пункт

Маркеры красные, жирные, увеличенные

Изменения в ::marker

Стилизация самого маркера

HTML

```
<ul class="styled-list">  
  <li>Первый пункт</li>  
  <li>Второй пункт</li>  
  <li>Третий пункт</li>  
  <li>Четвертый пункт</li>  
</ul>
```

Пример list-style-type

CSS

```
.disc { list-style-type: disc; } /*  
Заполненный круг */  
.circle { list-style-type: circle; } /*  
Пустой круг */  
  
li {  
  margin: 5px 0;  
  padding: 5px;  
  background: #f0f0f0;  
}
```

Пример list-style-type

- disc (круг)
- circle (пустой круг)

Изменения в list-style-type

Тип маркера

HTML

```
<ul class="disc">  
  <li>disc (круг)</li>  
</ul>  
  
<ul class="circle">  
  <li>circle (пустой круг)</li>  
</ul>
```

Пример list-style-image

CSS

```
ul {  
  list-style-image: url('https://cdn-icons-  
png.flaticon.com/16/1828/1828884.png');  
}  
li { margin: 5px 0; }
```

- ★ Пункт 1
- ★ Пункт 2
- ★ Пункт 3

Изменения в list-style-image

Картинка вместо маркера

HTML

```
<ul class="image-list">  
  <li>Пункт со звездой</li>  
  <li>Пункт со звездой</li>  
  <li>Пункт со звездой</li>  
</ul>
```

Практическое применение

Документация

Автоматическая
нумерация
разделов и
подразделов с
помощью
счётчиков

Публикации

Многоколоночная
верстка для
газетных и
журнальных
макетов

Современный дизайн

Необычные
формы через
clip-path и
маски для
визуального
выделения

UI-компоненты

Стилизированные
списки для
интерфейсов и
пользовательских
форм

Сравнение свойств

clip-path mask

- Создает геометрические формы
- Поддержка базовых фигур
- SVG-контуры через path()
- Анимация не всегда поддерживается
- Использует изображения и градиенты
- Гибкость в дизайне
- Анимация через градиенты
- Требуется прозрачность в изображении

Благодарим за внимание!

Ключевые навыки
шаг
Управление
нумерацией,
многоколоночная
верстка, создание
нестандартных форм

Следующий

Практика
применения этих
техник в реальных
проектах

Документация

MDN Web Docs —
полное описание
всех CSS-свойств

Докладчики: Барабанов Илья, Волков Иван

Блок 4. Практика: Задания для самостоятельной работы

Согласно требованиям, здесь мы готовим заготовки и описываем задачи на 5–15 минут. Ответы к ним в репозиторий добавлять не нужно, но вы должны уметь их решить «без шпаргалок».

Задание №1: "Фигурные изображения"

Задача: Используя свойство `clip-path`, превратите обычное квадратное изображение в восьмиугольник (октагон).

Заготовка: Создайте файл `task1.html` с тегом `<img>`.

Требование: Использовать функцию `polygon()`.

Время: 10 минут.

Задание №2: "Газетная колонка"

Задача: Реализовать адаптивную раскладку текста. Текст должен автоматически разбиваться на колонки шириной не менее 250px, а между колонками должна появиться разделительная линия.

Заготовка: Файл `task2.html` с длинным текстом в теге `<article>`.

Требование: Использовать короткую запись `columns` и свойство `column-rule`.

Время: 5 минут.

Задание №3: "Автоматическое оглавление"

Задача: Настроить стили так, чтобы перед каждым заголовком `<h2>` автоматически выводился номер раздела в формате "Раздел №".

Заготовка: Файл `task3.html` с несколькими заголовками `<h2>`.

Требование: Использовать `counter-reset`, `counter-increment` и псевдоэлемент `::before`.

Время: 15 минут.